

Lecture 12: Givens rotations and Gram-Schmidt

Jamie Haddock

Table of contents

1	Givens rotations	1
1.1	QR by Givens rotations	2
2	Gram-Schmidt	3
2.1	Summary	5

1 Givens rotations

Householder reflections zero out an entire column below the diagonal at once. Sometimes we want to zero out a *single* entry.

Definition: Givens rotation

A **Givens rotation** is a matrix of the form

$$\mathbf{G} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \quad c^2 + s^2 = 1.$$

Exercise:

When would zeroing one entry at a time be advantageous?

Answer:

When there is sparsity already present in the matrix! A classic example is a **Hessenberg** matrix which has only nonzeros in on or above the main diagonal and in the diagonal directly below the main diagonal.

When zeroing out a single entry in a given column, we just need a 2x2 matrix that uses one nonzero entry to zero out the other one. We can embed this matrix into an identity matrix so that it only acts on the rows with these two nonzero entries.

Theorem: Givens rotation

A Givens rotation, $\mathbf{G} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$ is orthogonal. Moreover, for $\mathbf{x} = \begin{bmatrix} a \\ b \end{bmatrix}$ and $r = \sqrt{a^2 + b^2} \neq 0$, taking $c = a/r$ and $s = b/r$ gives

$$\mathbf{G}\mathbf{x} = \begin{bmatrix} r \\ 0 \end{bmatrix}.$$

Proof:

$$\mathbf{G}^\top \mathbf{G} = \begin{bmatrix} c^2 + s^2 & 0 \\ 0 & s^2 + c^2 \end{bmatrix} = \mathbf{I}, \text{ and } \mathbf{G}\mathbf{x} = \begin{bmatrix} (a^2 + b^2)/r \\ (-ba + ab)/r \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}.$$

```
%%julia
using LinearAlgebra
```

```
x = [3., 4.]
r = hypot(x...)
c, s = x[1]/r, x[2]/r
G = [c s; -s c]
G*x
```

```
2-element Vector{Float64}:
 5.0
-4.440892098500626e-16
```

To act on rows i and j of an $m \times n$ matrix $\mathbf{A}$, embed the columns of $\mathbf{G}$ in to the i th and j th columns of an identity matrix. Only those two rows of $\mathbf{A}$ change.

```
%%julia
x = [0, 0, 3., 0, 4., 0]
G = Matrix{Float64}(I, 6, 6)
G[3,3] = c
G[3,5] = s
G[5,3] = -s
G[5,5] = c
```

```
G
```

```
6×6 Matrix{Float64}:
 1.0  0.0  0.0  0.0  0.0  0.0
 0.0  1.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.6  0.0  0.8  0.0
 0.0  0.0  0.0  1.0  0.0  0.0
 0.0  0.0 -0.8  0.0  0.6  0.0
 0.0  0.0  0.0  0.0  0.0  1.0
```

```
%%julia
G*x
```

```
6-element Vector{Float64}:
 0.0
 0.0
 5.0
 0.0
-4.440892098500626e-16
 0.0
```

1.1 QR by Givens rotations

In the worst case (fully dense matrix), we use rotations which use the entry above to eliminate the entry below all through the lower-triangular part of the matrix.

With $\mathbf{G}_1, \mathbf{G}_2, \dots, \mathbf{G}_N$ the (embedded) rotations used,

$$\mathbf{G}_N \cdots \mathbf{G}_1 \mathbf{A} = \mathbf{R} \quad \Rightarrow \quad \mathbf{Q} = \mathbf{G}_1^\top \cdots \mathbf{G}_N^\top.$$

```

%%julia
"""
    qrgivens(A)
QR factorization by Givens rotations. Returns Q and R.
"""
function qrgivens(A)
    m,n = size(A)
    Q = Matrix{Float64}(I,m,m)
    R = copy(A)
    for j in 1:n
        for i in m:-1:j+1
            a,b = R[i-1,j], R[i,j]
            r = hypot(a,b)
            r == 0 && continue
            c,s = a/r, b/r
            G = [c s; -s c]
            R[i-1:i,:] = G*R[i-1:i,:]
            Q[:,i-1:i] = Q[:,i-1:i]*G'
        end
    end
    return Q, triu(R)
end

```

qrgivens

```

%%julia
A = rand(1:9,6,4)
Q,R = qrgivens(A)
@show opnorm(Q*R - A)
@show opnorm(Q'*Q - I);

```

```

opnorm(Q * R - A) = 5.803502081424711e-15
opnorm(Q' * Q - I) = 5.736192209707644e-16

```

Theorem:

QR factorization of a dense $m \times n$ matrix by Givens rotations takes $\sim (3mn^2 - n^3)$ flops.

For a square matrix this is $2n^3$, about $1.5\times$ the cost of Householder ($\frac{4}{3}n^3$). So why use Givens?

Givens rotations exploit **structure**. If a column already has few nonzeros below the diagonal, we only rotate away those entries and we avoid **fill-in**!

For example, an upper Hessenberg matrix has one subdiagonal entry per column, so its QR factorization takes $n - 1$ rotations and only $O(n^2)$ flops.

2 Gram-Schmidt

Householder reflections and Givens rotations construct a QR factorization with square $\mathbf{Q}$ formed as a product of orthogonal matrices. Gram-Schmidt computes the thin QR factorization $\mathbf{A} = \hat{\mathbf{Q}}\hat{\mathbf{R}}$ by orthogonalizing the columns of $\mathbf{A}$ one at a time. This can be advantageous when $\mathbf{A}$ is very thin and tall.

i Recall: Try it by hand

Let

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 3 \\ 2 & 3 & 3 \\ 1 & 3 & 6 \end{bmatrix}.$$

Working one column at a time:

1. Normalize $\mathbf{a}_1$ to get $\mathbf{q}_1$. Record the scaling factor r_{11} .
 2. Subtract from $\mathbf{a}_2$ its component along $\mathbf{q}_1$, then normalize to get $\mathbf{q}_2$. Record r_{12} and r_{22} .
 3. Subtract from $\mathbf{a}_3$ its components along $\mathbf{q}_1$ and $\mathbf{q}_2$, then normalize to get $\mathbf{q}_3$. Record r_{13}, r_{23}, r_{33} .
- Check that $\mathbf{A} = \mathbf{Q}\mathbf{R}$, where $\mathbf{R}$ is the upper triangular matrix of the numbers you recorded.

At step k , start with $\mathbf{v}^{(0)} = \mathbf{a}_k$ and orthogonalize against previous columns of $\mathbf{Q}$ one at a time:

$$\mathbf{v}^{(j)} = \mathbf{v}^{(j-1)} - r_{jk} \mathbf{q}_j, \quad j = 1, \dots, k-1.$$

Then $r_{kk} = \|\mathbf{v}^{(k-1)}\|$ and $\mathbf{q}_k = \mathbf{v}^{(k-1)} / r_{kk}$.

Now, how do we compute r_{jk} ?

- **Classical** (CGS): $r_{jk} = \mathbf{q}_j^\top \mathbf{a}_k$ (the original column)
- **Modified** (MGS): $r_{jk} = \mathbf{q}_j^\top \mathbf{v}^{(j-1)}$ (the partially orthogonalized vector)

These are identical in exact arithmetic, since $\mathbf{q}_j \perp \mathbf{q}_1, \dots, \mathbf{q}_{j-1}$.

```
%%julia
function cgs(A)
    m,n = size(A)
    Q = zeros(m,n)
    R = zeros(n,n)
    for k in 1:n
        v = A[:,k]
        for j in 1:k-1
            R[j,k] = Q[:,j]'*A[:,k]
            v -= R[j,k]*Q[:,j]
        end
        R[k,k] = norm(v)
        Q[:,k] = v/R[k,k]
    end
    return Q, R
end

function mgs(A)
    m,n = size(A)
    Q = zeros(m,n)
    R = zeros(n,n)
    for k in 1:n
        v = A[:,k]
        for j in 1:k-1
            R[j,k] = Q[:,j]'*v
            v -= R[j,k]*Q[:,j]
        end
        R[k,k] = norm(v)
        Q[:,k] = v/R[k,k]
    end
    return Q, R
end
```

```

        end
        R[k,k] = norm(v)
        Q[:,k] = v/R[k,k]
    end
    return Q, R
end

```

mgs (generic function with 1 method)

Gram-Schmidt costs $\sim 2mn^2$ flops, the same as Householder QR, but it is not backward stable. The danger is **loss of orthogonality**: when columns of $\mathbf{A}$ are nearly dependent, subtracting projections causes cancellation and the computed $\hat{\mathbf{Q}}$ is not orthonormal.

We test with a Lauchli matrix, whose columns are nearly parallel.

```

%%julia
= 1e-8
A = [ones(1,3); *I(3)]

Qc,_ = cgs(A)
Qm,_ = mgs(A)
Qh   = Matrix(qr(A).Q)

@show opnorm(Qc'*Qc - I)
@show opnorm(Qm'*Qm - I)
@show opnorm(Qh'*Qh - I);

```

```

opnorm(Qc' * Qc - I) = 0.5000000000000001
opnorm(Qm' * Qm - I) = 8.164965844469921e-9
opnorm(Qh' * Qh - I) = 4.486834531857287e-16

```

Note:

Loss of orthogonality in CGS can be $O(1)$. In MGS it grows like $\varepsilon_{\text{mach}} \kappa(\mathbf{A})$. Householder QR keeps $\hat{\mathbf{Q}}$ orthonormal to $O(\varepsilon_{\text{mach}})$ regardless of $\kappa(\mathbf{A})$.

2.1 Summary

Method	Flops ($m \times n$)	Stable?	Best for
Householder	$2mn^2 - \frac{2}{3}n^3$	Yes	dense matrices
Givens	$3mn^2 - n^3$	Yes	sparse / structured matrices
Gram-Schmidt	$2mn^2$	No (MGS: only mildly)	building orthonormal bases one vector at a time